

Prévision du Niveau du Réservoir

Projet WADI — Hackathon IA

Université Clermont Auvergne

UFR de Mathématiques

Rapport de Projet

Contexte	Hackathon IA — Actemium
Dataset	WADI (<i>Water Distribution</i>) — 14 jours
Date	Avril 2026
Objectif	Forecasting multivarié temporel du niveau réservoir (3_LT_001_PV)

Étudiants

Stowe AGBESSI
Victoria BOUCHET
Karen Mariani LOCKO KENGUE
Abdoulaye Brahim MAHAMAT
Miroslav PENKOV
Merlin SIPAHIMALAN

Enseignants référents

Nourdine AZZAOU
Erwan SAINT LOUBERT BIE
Boris NECTOUX

Table des matières

1	Introduction et contexte	3
1.1	Présentation du projet	3
1.2	Architecture du système WADI	3
1.3	Structure du rapport	3
2	Description des données	4
2.1	Vue d'ensemble	4
2.2	Familles de variables	5
2.3	Variables cibles : les colonnes LT	5
3	Analyse Exploratoire des Données (EDA)	6
3.1	Méthodologie de nettoyage	6
3.1.1	Reconstruction de l'index temporel	6
3.1.2	Traitement des valeurs manquantes	6
3.1.3	Gestion des lignes dupliquées	6
3.1.4	Dimensions après nettoyage	7
3.2	Détection des valeurs aberrantes	7
3.3	Analyse des corrélations	7
3.4	Analyse en Composantes Principales (ACP)	8
4	Protocole expérimental et modèles de référence (baselines)	9
4.1	Protocole commun à tous les modèles	9
4.2	Modèles statistiques classiques — ARIMA et ETS	10
4.2.1	Description des modèles	10
4.2.2	Résultats	11
4.2.3	Analyse des résultats	11
5	Comparaison des modèles et justification du choix de XGBoost	11
5.1	Modèles envisagés	11
5.2	Pourquoi XGBoost a été privilégié	12
5.3	Résultats comparatifs	12
5.4	Place des modèles LSTM, GRU, TCN et Transformer	13
6	XGBoost avec optimisation bayésienne des hyperparamètres	13
6.1	Approche et feature engineering temporel	13
6.2	Optimisation des hyperparamètres — Optuna (TPE)	14
6.3	Résultats d'évaluation	15
6.4	Regard critique	15

7	Modèles de deep learning temporel	15
7.1	Protocole d'évaluation	15
7.2	Modèle TCN (Temporal Convolutional Network)	16
7.3	Modèle LSTM (Long Short-Term Memory)	16
7.4	Résultats préliminaires ARIMA / ETS pour référence	16
7.5	Difficultés rencontrées et limites expérimentales	17
7.5.1	Volume et complexité des données	17
7.5.2	Coût computationnel	17
7.5.3	Choix de la variable cible	17
7.5.4	Contraintes organisationnelles	17
8	Discussion et perspectives	18
8.1	Bilan des travaux	18
8.2	Limites identifiées	18
8.3	Recommandations pour la suite	19
	Annexes	20

1 Introduction et contexte

1.1 Présentation du projet

Ce rapport présente les travaux réalisés dans le cadre du Hackathon IA sur le jeu de données **WADI** (*Water Distribution*). WADI est un banc d'essai industriel de distribution d'eau, instrumenté avec des capteurs physiques et des actionneurs (pompes, vannes) pilotés par un système SCADA (*Supervisory Control and Data Acquisition*).

L'**objectif principal** est de prédire le niveau du réservoir principal (3_LT_001_PV) à des horizons de 5, 10 et 30 minutes, afin d'anticiper les risques de débordement et d'optimiser les cycles de pompage.

Ce projet mobilise plusieurs approches complémentaires : une analyse exploratoire approfondie des données, des modèles statistiques classiques (ARIMA, ETS), des modèles de *machine learning* (XGBoost) et des modèles de deep learning temporel (TCN, LSTM, GRU).

1.2 Architecture du système WADI

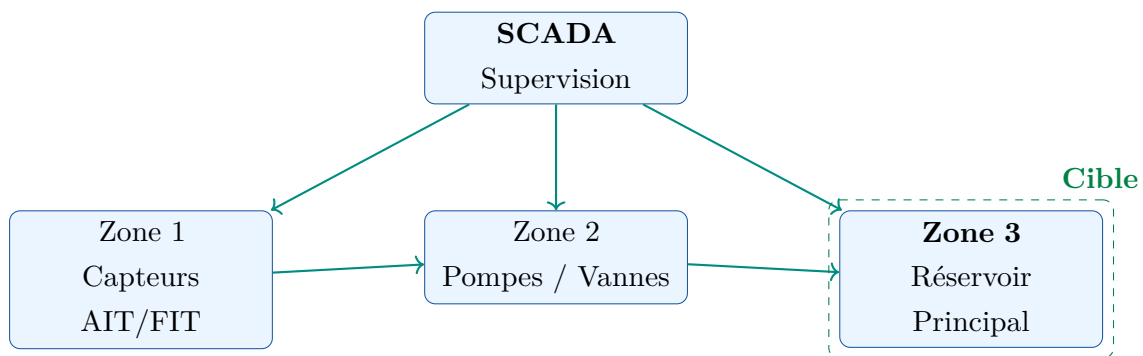


FIGURE 1 – Architecture simplifiée du système WADI et identification de la zone cible

1.3 Structure du rapport

Ce rapport est organisé en cinq grandes parties :

1. **Description des données** — vue d'ensemble, familles de variables, statistiques descriptives des cibles.
2. **Analyse exploratoire (EDA)** — nettoyage, corrélations, réduction de dimensionnalité.
3. **Protocole expérimental et baselines** — découpage temporel, métriques, modèles statistiques (ARIMA, ETS).
4. **Comparaison des modèles et choix de XGBoost** — justification de l'approche, résultats comparatifs.
5. **XGBoost avec optimisation bayésienne** — feature engineering, tuning Optuna, évaluation finale.

6. **Modèles de deep learning** — TCN, LSTM, GRU : architecture, résultats, difficultés.
7. **Discussion et perspectives** — bilan, limites, recommandations.

2 Description des données

2.1 Vue d'ensemble

TABLE 1 – Caractéristiques générales du jeu de données WADI

Caractéristique	Valeur
Nombre de lignes brutes	784 571
Nombre de colonnes brutes	130
Fréquence d'échantillonnage	1 mesure / seconde
Durée totale couverte	25/09/2017 au 07/10/2017 ($\approx$ 13 jours)
Colonnes après nettoyage	121
Variables cibles (LT)	4 colonnes
Variable cible principale	3_LT_001_PV

Coupure dans les données

Une interruption de $\approx$ **3 jours** a été détectée entre le 29/09 et le 02/10/2017, correspondant à un arrêt physique du système SCADA (saut de 264 001 unités dans la colonne **Row**). Cette coupure a été documentée et prise en compte lors de la construction des séquences temporelles pour éviter de créer des fenêtres glissantes qui enjambent cette période.

2.2 Familles de variables

TABLE 2 – Résumé des familles de variables WADI

Préfixe	Nb colonnes	Description
AIT	18	Qualité de l'eau (pH, chlore, turbidité, conductivité)
FIT	5	Débits entrants/sortants des réservoirs
LT	4	Niveaux des réservoirs — variable(s) cible(s)
LS	17	Alarmes de niveau haut/bas (binaire)
MV	20	Commandes ON/OFF des vannes motorisées
FIC	18	Boucles PID de débit (mesure, consigne, commande)
PIC	3	Boucles PID de pression
PIT / DPIT	5	Pression locale et différentielle
FQ	6	Volume total d'eau écoulé (compteur cumulatif)
P_STATUS	17	État ON/OFF des pompes

2.3 Variables cibles : les colonnes LT

Quatre colonnes mesurent le niveau des réservoirs du système. Leur analyse statistique révèle des comportements très différents :

TABLE 3 – Statistiques descriptives des variables de niveau (LT)

Variable	Moyenne (m)	Écart-type	Min	Max
1_LT_001_PV	56.37	8.85	38.81	71.62
2_LT_001_PV	69.77	0.65	68.51	71.55
2_LT_002_PV	75.41	3.47	65.70	82.98
3_LT_001_PV	65.17	1.22	63.96	69.17

La variable 3_LT_001_PV (niveau du réservoir principal — Zone 3) est retenue comme **cible principale** pour la modélisation, conformément aux consignes du projet. Sa faible variabilité ($\sigma = 1.22$ m) représente un défi particulier : un modèle naïf de persistance est difficile à surpasser sur cette cible, ce qui rend l'évaluation plus exigeante.

3 Analyse Exploratoire des Données (EDA)

3.1 Méthodologie de nettoyage

3.1.1 Reconstruction de l'index temporel

Point technique important — Format de la colonne Time

La colonne `Time` du fichier brut est au format `MM:SS` (minutes :secondes), **non** `HH:MM`. Elle ne code que 3 600 valeurs distinctes par heure, sans information sur l'heure elle-même. La reconstruction naïve via `pd.to_datetime(Date + Time)` produit des centaines de milliers de doublons artificiels.

Solution adoptée : l'index temporel est reconstruit à partir de la colonne `Row` (compteur séquentiel en secondes depuis le démarrage) :

$$\text{Datetime} = 2017-09-25\ 00:00:00 + (\text{Row} - 1) \times 1\text{ s}$$

Cette approche donne un index sans aucun doublon et cohérent avec la fréquence d'échantillonnage réelle de **1 mesure/seconde**.

3.1.2 Traitement des valeurs manquantes

TABLE 4 – Bilan du traitement des valeurs manquantes

Colonne(s)	% manquants	Action
2_LS_001_AL, 2_LS_002_AL	100%	Suppression
2_P_001_STATUS, 2_P_002_STATUS	100%	Suppression
2_MCV_007_CO, 2_PIC_003_SP	Constantes	Suppression (variance nulle)
1_AIT_002_PV et autres	< 0.01%	Interpolation linéaire (limit=5 pas)
Variables catégorielles (STATUS, AL)	Résiduel	Forward-fill (dernier état connu)
Valeurs manquantes résiduelles		9 (lacunes > 5 s conservées)

3.1.3 Gestion des lignes dupliquées

Le dataset contient **580 452 lignes** où toutes les valeurs sont identiques à la ligne précédente ($\approx 74\%$ des données). Dans un contexte SCADA industriel, ce phénomène de « gel de capteur » est parfaitement normal : lorsqu'une grandeur physique ne varie pas, le système répète la dernière mesure.

Ces lignes **ne sont pas supprimées**, car leur suppression détruirait la régularité temporelle

de la série à 1 Hz, indispensable au bon fonctionnement des modèles séquentiels (LSTM, GRU, fenêtres glissantes XGBoost).

3.1.4 Dimensions après nettoyage



3.2 Détection des valeurs aberrantes

La méthode de Tukey (règle $IQR \times 1.5$) a été appliquée sur l'ensemble des variables numériques. Les cinq variables présentant le plus fort taux de valeurs extrêmes sont :

TABLE 5 – Top 5 des variables avec le plus d'outliers (méthode IQR)

Variable	Nb outliers	% outliers
2_FIT_001_PV	190 228	24.25%
3_AIT_002_PV	179 875	22.93%
3_FIT_001_PV	165 392	21.08%
2_PIC_003_CD	148 348	18.91%
2_PIC_003_PV	144 116	18.37%

Interprétation industrielle

Dans un système de distribution d'eau, les valeurs extrêmes des débits (FIT) et pressions (PIC) correspondent typiquement à des **événements physiques réels** : ouverture/fermeture brusque d'une vanne, démarrage de pompe, transitoire hydraulique. Ces valeurs sont **conservées** et constituent des **signaux informatifs** pour prédire les variations du niveau.

3.3 Analyse des corrélations

L'analyse de corrélation de Pearson entre toutes les variables numériques et la cible 3_LT_001_PV révèle les variables les plus informatives :

TABLE 6 – Top 7 variables corrélées avec 3_LT_001_PV

Variable	Pearson	Interprétation
2_FIC_401_CO	0.641	Commande PID débit zone 2→3
2_FIC_101_CO	0.529	Commande PID débit secondaire
3_FIT_001_PV	0.451	Débit entrant réservoir zone 3
2_MCV_301_CO	0.448	Commande vanne principale
2_MCV_601_CO	0.402	Commande vanne secondaire
2_FIT_002_PV	0.373	Débit sortant zone 2
3_AIT_005_PV	0.353	Qualité eau zone 3

Interprétation

Les commandes de vannes (MCV) et les boucles de régulation de débit (FIC) sont les variables les plus corrélées avec le niveau du réservoir — ce qui est physiquement cohérent. Ces variables seront des features prioritaires pour la modélisation.

3.4 Analyse en Composantes Principales (ACP)

Une ACP a été réalisée sur les 63 variables numériques continues (hors colonnes LT) après standardisation, sur un sous-échantillon de 10% des données soit 78 457 observations.

TABLE 7 – Variance expliquée par les premières composantes principales

Composante	Variance expliquée (%)	Cumulé (%)
PC1	23.05	23.05
PC2	13.83	36.89
PC3	9.07	45.96
PC4	7.47	53.43
PC5	5.00	58.43
PC6	3.32	61.75
PC7	3.04	64.78
Pour atteindre 80% de variance		≈ 15 composantes

Conclusions de l'ACP

- Les 7 premières composantes n'expliquent que 65% de la variance — le jeu de données est **intrinsèquement de haute dimension**, sans structure basse-dimension dominante.
- La projection PC1/PC2 colorée par le niveau cible révèle une **structure non**

linéaire, ce qui justifie l’usage de modèles non linéaires (XGBoost, LSTM) plutôt qu’une régression linéaire simple.

- **PC1** est principalement portée par les variables de débit et de commande de vanne
- cohérent avec les résultats de corrélation.

4 Protocole expérimental et modèles de référence (bases)

4.1 Protocole commun à tous les modèles

Afin d’évaluer les performances des différents modèles de manière homogène, un protocole de comparaison rigoureux a été mis en place.

Découpage chronologique des données. Les données ont été séparées de manière **strictement chronologique** en trois ensembles, sans aucun mélange aléatoire, afin de simuler des conditions réalistes de prédiction et d’éviter toute fuite d’information (*data leakage*) :



TABLE 8 – Répartition des données après split temporel

Ensemble	Période	Lignes	Proportion
Train	25/09 → 04/10	142 839	70%
Validation	04/10 → 05/10	30 608	15%
Test	05/10 → 07/10	30 609	15%

Prétraitement commun. Les variables explicatives ont été préparées comme suit : suppression de la variable cible des features, conservation uniquement des variables numériques, imputation des valeurs manquantes résiduelles par propagation avant (*forward fill*).

Métriques d’évaluation Trois métriques principales ont été utilisées pour tous les modèles :

- **RMSE** (*Root Mean Squared Error*) :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

— **MAE** (*Mean Absolute Error*) :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

— **Skill Score** (comparaison à la baseline de persistance) :

$$SS = 1 - \frac{\text{RMSE}_{\text{modèle}}}{\text{RMSE}_{\text{baseline}}}$$

Un Skill Score positif indique une amélioration par rapport à la baseline naïve. Le critère de réussite fixé est une amélioration d'au moins **20%** en RMSE.

Modèle de référence — Baseline de persistance Le premier modèle de référence est un prédicteur naïf persistant :

$$\hat{y}(t) = y(t-1)$$

Ce modèle est particulièrement difficile à battre sur les données SCADA car les séries contiennent de longues périodes stables (74% de lignes identiques). Tout modèle plus complexe doit impérativement améliorer cette baseline pour être considéré comme pertinent.

4.2 Modèles statistiques classiques — ARIMA et ETS

4.2.1 Description des modèles

Modèles ARIMA Les modèles ARIMA (*AutoRegressive Integrated Moving Average*) et leurs extensions SARIMA et ARIMAX sont des modèles statistiques classiques pour la prévision des séries temporelles. Ils reposent sur trois composantes principales :

- une composante autorégressive (AR) qui modélise la dépendance aux valeurs passées,
- une composante de moyenne mobile (MA) qui capture l'impact des erreurs passées,
- une différenciation (I) permettant de rendre la série stationnaire.

Le modèle SARIMA étend ARIMA en intégrant une composante saisonnière explicite. Le modèle ARIMAX ajoute des variables exogènes afin d'intégrer des facteurs explicatifs externes.

Plusieurs configurations ont été testées : modélisation directe sur la série brute, transformation logarithmique pour les dynamiques multiplicatives, et décomposition STL (*Seasonal-Trend decomposition using LOESS*) sur la composante résiduelle.

Modèle Holt-Winters (ETS) Le modèle Holt-Winters repose sur une décomposition exponentielle en trois composantes : niveau, tendance et saisonnalité. Deux variantes ont été testées — additive et multiplicative — la première étant retenue, le signal présentant un comportement additif.

4.2.2 Résultats

TABLE 9 – Performances des modèles statistiques sur le set de test (3_LT_001_PV)

Modèle	RMSE	MAE	Skill Score
ARIMA	1.5186	0.9764	−27.81
ETS	1.5229	0.9788	−27.89

4.2.3 Analyse des résultats

Les résultats montrent des performances proches entre les deux modèles, ARIMA obtenant un léger avantage. Cependant, le **Skill Score fortement négatif** pour les deux modèles indique que leurs performances sont largement inférieures à celles de la baseline de persistance.

Ce résultat s’explique par plusieurs facteurs :

- La série temporelle présente une forte inertie (faibles variations entre deux instants consécutifs), ce qui rend le modèle naïf particulièrement difficile à battre.
- Les modèles ARIMA et ETS, bien qu’adaptés aux séries univariées, peinent à capturer la complexité du système industriel, notamment les interactions entre les différentes variables de commande.
- L’utilisation d’un grand nombre de variables exogènes sans sélection préalable introduit du bruit et dégrade les performances.
- Des contraintes matérielles importantes ont limité l’exploration des hyperparamètres : environ 30 minutes d’entraînement par run ARIMA, rendant toute grid search exhaustive impossible.

L’analyse visuelle des prédictions confirme ces observations : les deux modèles reproduisent globalement la tendance de la série, mais se comportent de manière proche d’un prédicteur naïf sur les phases stables — qui constituent l’essentiel des données.

5 Comparaison des modèles et justification du choix de XGBoost

Dans le cadre de ce projet, plusieurs familles de modèles ont été testées afin de prédire le niveau du réservoir. L’objectif n’était pas uniquement d’obtenir le meilleur score possible, mais de sélectionner une approche cohérente avec la nature industrielle des données, la contrainte temporelle du hackathon et les critères d’évaluation attendus.

5.1 Modèles envisagés

Outre la baseline de persistance et les modèles statistiques (ARIMA, ETS) présentés précédemment, deux familles supplémentaires ont été évaluées.

Modèles ML sur données tabulaires Une régression locale linéaire et XGBoost ont été testés avec des variables retardées (*lags*). La dynamique temporelle est encodée explicitement dans les features, car XGBoost ne possède pas de mémoire interne : il utilise les valeurs passées du niveau, des débits, des commandes de vannes et des états de pompes pour prédire le niveau futur.

Modèles de deep learning Des architectures LSTM, GRU, TCN et Transformer temporel ont également été prévues et implémentées. Ces modèles apprennent directement à partir de fenêtres de mesures successives et captent les effets retardés des actionneurs sans feature engineering manuel. Ils sont présentés en section 7.

5.2 Pourquoi XGBoost a été privilégié

XGBoost a été retenu comme modèle principal car il offre un bon compromis entre performance, robustesse, rapidité d’entraînement et explicabilité.

- **Non-linéarité** : contrairement à une régression linéaire, XGBoost modélise des relations non linéaires entre les variables, ce qui est essentiel dans le contexte WADI où l’évolution du niveau dépend d’interactions complexes entre débits, vannes et boucles de régulation.
- **Rapidité** : dans le cadre d’un hackathon, la capacité à tester rapidement plusieurs configurations (variables cibles, horizons, ensembles de features) est déterminante. Les modèles LSTM et GRU sont beaucoup plus coûteux à entraîner sur un dataset de $\approx 785\,000$ lignes.
- **Interprétabilité** : XGBoost permet d’analyser l’importance des variables ou d’utiliser des méthodes comme SHAP pour vérifier la cohérence physique des décisions, ce qui est plus difficile avec les modèles de deep learning.

5.3 Résultats comparatifs

TABLE 10 – Comparaison des modèles pour la cible 1_LT_001_PV à horizon 10 minutes

Modèle	MAE	RMSE	MAPE	Skill vs persistence
XGBoost	5.80	6.78	12.32%	+21.5%
Persistence	6.88	8.64	14.16%	0.0%
Régression locale	15.30	16.79	31.70%	−94.4%
Kalman	50.36	94.41	100.20%	−1011.0%

Cible 1_LT_001_PV — horizon 10 minutes Sur 1_LT_001_PV, XGBoost obtient un RMSE de 6.78 contre 8.64 pour la baseline de persistence, soit une amélioration de **21.5%** — au-dessus du critère de réussite de 20%.

TABLE 11 – Comparaison des modèles pour la cible 3_LT_001_PV à horizon 10 minutes

Modèle	MAE	RMSE	MAPE	Skill vs persistence
Persistence	0.0848	0.1446	0.13%	0.0%
XGBoost	0.1435	0.2402	0.22%	−66.1%
Régression locale	0.5408	0.6761	0.84%	−367.7%
Kalman	2.6026	3.4174	4.04%	−2275.6%

Cible 3_LT_001_PV — horizon 10 minutes Sur 3_LT_001_PV, la persistence reste meilleure en score global. Ce résultat s’explique par la faible variabilité de cette cible ($\sigma = 1.22\text{m}$) : recopier la dernière valeur est une stratégie très compétitive lorsque le signal varie peu. Pour juger correctement l’apport de XGBoost, il faut analyser les **phases de transition** (démarrages de pompe, ouvertures de vanne), où les variables exogènes deviennent plus informatives.

5.4 Place des modèles LSTM, GRU, TCN et Transformer

Ces modèles ont été considérés comme des pistes d’amélioration et de comparaison. Ils apprennent directement à partir de séquences temporelles sans feature engineering manuel, ce qui les rend théoriquement plus expressifs. Cependant, dans notre contexte, ils présentent plusieurs limites pratiques : temps d’entraînement élevé (jusqu’à 1 heure pour 20 époques), sensibilité aux hyperparamètres, et nécessité d’une gestion rigoureuse des fenêtres autour des coupures temporelles. Leurs résultats sont présentés en section 7.

6 XGBoost avec optimisation bayésienne des hyperparamètres

6.1 Approche et feature engineering temporel

L’objectif est de prédire la variable 3_LT_001_PV à partir des mesures passées issues des différents capteurs. Cette tâche est formulée comme un problème de **régression supervisée sur des données temporelles multivariées**.

XGBoost n’ayant pas de mémoire intrinsèque, il est nécessaire d’encoder explicitement la dynamique temporelle dans les features.

Pour chaque lag $k \in \{1, 2, 3, 5, 10, 20, 60\}$ secondes :

$$\text{lag}_k(t) = x(t - k)$$

Pour des fenêtres glissantes de taille $w \in \{5, 10, 20, 60\}$ secondes, avec décalage d'un pas pour éviter toute fuite de données :

$$\text{roll_mean}_w(t) = \frac{1}{w} \sum_{i=1}^w x(t-i) \quad \text{roll_std}_w(t) = \text{std}(x(t-1), \dots, x(t-w))$$

Résultat : 131 features au total après suppression des lignes avec NA induits par les lags (60 lignes supprimées).

6.2 Optimisation des hyperparamètres — Optuna (TPE)

L'optimisation bayésienne par algorithme TPE (*Tree-structured Parzen Estimator*) via la librairie Optuna a été utilisée à la place d'une grid search classique, pour explorer l'espace d'hyperparamètres de façon guidée et efficace.

L'espace de recherche inclut les paramètres de régularisation L1 et L2 (pour éviter l'overfitting), la profondeur des arbres, le learning rate, ainsi que les paramètres de sous-échantillonnage. L'objectif est de minimiser le RMSE sur le jeu de validation. Chaque configuration est évaluée avec un mécanisme d'*early stopping* (patience = 50 rounds).

TABLE 12 – Meilleurs hyperparamètres trouvés par Optuna (15 trials)

Hyperparamètre	Valeur optimisée	Rôle
max_depth	6	Profondeur maximale des arbres
eta (learning rate)	0.1859	Pas d'apprentissage
subsample	0.92399	Fraction de lignes par arbre
colsample_bytree	0.93207	Fraction de colonnes par arbre
gamma	0.18407	Gain minimum pour créer un nœud
min_child_weight	10	Somme minimale de poids par feuille
lambda	≈ 0	Régularisation L2
alpha	≈ 0	Régularisation L1
tree_method	hist	Algorithme de construction
objective	reg:squarederror	Fonction de perte MSE à minimiser
eval_metric	rmse	Métrique surveillée pendant l'entraînement
seed	42	Graine aléatoire pour la reproductibilité

6.3 Résultats d'évaluation

Performances du modèle XGBoost sur le set de test — 3_LT_001_PV

Métrique	Valeur	Interprétation
RMSE	0.0510	Erreur quadratique moyenne
MAE	0.0373	Erreur absolue moyenne
R^2	0.9982	99.82% de la variance du niveau expliquée

Ces métriques montrent une nette amélioration par rapport aux baselines ARIMA et ETS. Les prédictions et les données réelles sont très proches ; graphiquement, les courbes réelle et prédite se superposent, ce qui constitue un fort argument visuel en faveur de l'approche.

On note cependant que 3_LT_001_PV est peu variable, ce qui favorise mécaniquement un R^2 élevé. La forte autocorrélation du signal peut partiellement expliquer ce résultat sans exclure un risque de *data leakage*, qui n'a pas pu être vérifié formellement par manque de temps.

6.4 Regard critique

- **Nombre de trials insuffisant** : 15 trials Optuna sont peu pour explorer pleinement l'espace des hyperparamètres. Un minimum de 50 trials serait recommandé, mais les contraintes matérielles (machines personnelles) ont rendu cela impraticable.
- **Preuve de concept** : l'évaluation a été conduite sur 3_LT_001_PV uniquement. Ce choix est volontaire — il s'agit de démontrer la faisabilité et le potentiel de l'approche, non d'un résultat final généralisé.
- **Risque de data leakage** : le R^2 très élevé (0.9982) peut s'expliquer par la forte autocorrélation du signal, mais ne permet pas d'exclure un data leakage sans vérification approfondie, non réalisée faute de temps.

7 Modèles de deep learning temporel

7.1 Protocole d'évaluation

Tous les modèles de deep learning ont été évalués selon le même protocole chronologique (70/15/15) et les mêmes métriques (RMSE, MAE, Skill Score) que les modèles précédents.

Le prétraitement conserve l'ensemble des variables disponibles (y compris les covariables), sans sélection préalable. Deux stratégies de prédiction ont été comparées :

- **Directe** : prédiction de plusieurs pas de temps simultanément — stratégie privilégiée pour sa stabilité.
- **Réursive** : prédiction pas à pas en réinjectant les sorties comme entrées — introduit une accumulation d'erreur.

7.2 Modèle TCN (Temporal Convolutional Network)

Le TCN est un réseau de neurones convolutif adapté aux séries temporelles. Il repose sur :

- des **convolutions causales** (respect de la temporalité),
- des **dilatations exponentielles** (2^i), permettant de capturer des dépendances à différentes échelles temporelles,
- des **connexions résiduelles** pour stabiliser l'entraînement.

L'architecture implémentée repose sur plusieurs blocs résiduels avec des taux de dilatation croissants. À la sortie, seul le dernier pas de temps est utilisé, suivi d'un petit réseau dense pour stabiliser les prédictions en multi-step. Le TCN présente l'avantage de couvrir un historique temporel très large, ce qui est pertinent pour capturer les effets différés des actionneurs.

7.3 Modèle LSTM (Long Short-Term Memory)

Le LSTM est un réseau de neurones récurrent conçu pour capturer les dépendances à long terme. L'architecture retenue est composée de :

- deux couches LSTM (64 puis 32 unités),
- des couches de dropout pour limiter le sur-apprentissage,
- une couche dense de sortie.

Des variantes ont également été testées : LSTM bidirectionnel et architecture GRU (*Gated Recurrent Unit*), plus rapide à entraîner. Le GRU est une simplification du LSTM qui réduit le nombre de paramètres tout en conservant la capacité à modéliser des dépendances temporelles.

Le modèle VARMA a aussi été implémenté : il prend en compte les quatre variables cibles simultanément (modélisation multivariée complète), au prix d'un coût computationnel encore plus élevé.

7.4 Résultats préliminaires ARIMA / ETS pour référence

Pour rappel des ordres de grandeur, les premiers résultats comparatifs obtenus par les modèles statistiques sont les suivants :

Modèle	RMSE	MAE	Skill Score
ARIMA	1.5186	0.9764	-27.81
ETS	1.5229	0.9788	-27.89

Les modèles de deep learning sont susceptibles d'améliorer ces performances grâce à leur capacité à modéliser des dépendances multivariées complexes.

7.5 Difficultés rencontrées et limites expérimentales

7.5.1 Volume et complexité des données

Le jeu de données comporte $\approx 780\,000$ observations et plus de 120 variables, dont plus de 50 variables catégorielles (états d'actionneurs encodés en binaire). Cette forte dimensionnalité a posé plusieurs problèmes : augmentation significative du temps d'entraînement, difficulté à identifier les variables pertinentes et risque accru de sur-apprentissage.

Faute de temps, aucune sélection de variables exogènes n'a pu être réalisée. L'ensemble des covariables disponibles a donc été utilisé.

7.5.2 Coût computationnel

- ARIMA : ≈ 30 minutes par entraînement,
- TCN / LSTM : jusqu'à 1 heure pour 20 époques,
- ETS : relativement rapide.

La parallélisation via `joblib` entraînait une explosion de la consommation mémoire sur les machines personnelles, rendant toute grid search exhaustive impossible.

7.5.3 Choix de la variable cible

Deux approches étaient possibles : modéliser chaque variable LT séparément ou effectuer une prédiction multivariée. Par contrainte de temps, la variable `3_LT_001_PV` a été retenue comme cible unique. Une analyse plus approfondie aurait pu révéler que certaines variables présentent une variance plus favorable à la prédiction.

7.5.4 Contraintes organisationnelles

L'équipe était répartie sur trois classes aux emplois du temps différents, avec un seul créneau hebdomadaire dédié au projet, limitant la coordination et les itérations. Ces contraintes ont eu un impact direct sur la cohérence globale et la capacité à prendre des décisions techniques concertées.

8 Discussion et perspectives

8.1 Bilan des travaux

TABLE 13 – Synthèse des travaux réalisés

Volet	Statut	Résultat
Chargement & index temporel	✓	Bug format MM:SS identifié et corrigé via Row
Nettoyage des données	✓	9 colonnes supprimées, 9 NA résiduels
Analyse des LT	✓	4 réservoirs analysés, profils journaliers caractérisés
Corrélations & ACP	✓	Variables FIC/MCV identifiées, non-linéarité confirmée
Baselines (persistence, ARIMA, ETS)	✓	ARIMA RMSE = 1.52, Skill Score −27.8
Comparaison modèles ML	✓	XGBoost +21.5% vs persistence sur 1_LT_001_PV
Feature engineering	✓	131 features (lags + rolling stats)
Tuning Optuna (XGBoost)	✓	RMSE = 0.051, MAE = 0.037, $R^2 = 0.9982$
Modèles DL (TCN, LSTM, GRU)	✓	Implémentés et évalués (résultats en cours de consolidation)
Horizons 10 / 30 min (DL)	Partiel	Feature engineering étendu prévu

8.2 Limites identifiées

- **Coupure de 3 jours** : la coupure entre le 29/09 et le 02/10 réduit la durée effective d'entraînement et peut biaiser les rolling stats calculées au voisinage de la rupture.
- **Gel de capteur** : les 74% de lignes à valeurs identiques consécutives peuvent biaiser les métriques d'évaluation à la hausse — la baseline de persistence est artificiellement compétitive sur ces instants.
- **Absence de sélection de variables** : toutes les covariables ont été conservées, augmentant le bruit et le coût computationnel. Une étape de sélection (SHAP, importance XGBoost, corrélation) améliorerait la qualité des modèles.
- **Nombre de trials Optuna** : 15 trials sont insuffisants pour une exploration complète. Un minimum de 50 trials est recommandé.
- **Risque de data leakage** : le R^2 très élevé de XGBoost (0.9982) n'a pas pu être formellement exclu d'un data leakage ; une vérification approfondie est nécessaire.
- **Non-stationnarité potentielle** : les tests de stationnarité (ADF, KPSS) et l'analyse ACF/PACF n'ont pas été conduits de manière exhaustive.

8.3 Recommandations pour la suite

1. **Sélection de variables** : utiliser les valeurs SHAP ou l'importance XGBoost pour identifier les features les plus pertinentes et alléger les modèles.
2. **Augmenter les trials Optuna** : passer à 50–100 trials pour une exploration plus rigoureuse des hyperparamètres.
3. **Vérification du data leakage** : mettre en place un test formalisé pour s'assurer que les features de lags ne créent pas de fuite d'information.
4. **Horizons multiples** : reformuler le problème pour prédire $y(t+N)$ avec $N \in \{300, 600, 1800\}$ secondes (5, 10, 30 minutes à 1 Hz).
5. **Consolider les résultats DL** : finaliser l'évaluation TCN, LSTM et GRU avec un protocole de comparaison homogène et des visualisations qualitatives des prédictions.
6. **Analyse des phases transitoires** : évaluer les modèles spécifiquement sur les instants de démarrage de pompe ou d'ouverture de vanne, où l'apport des variables exogènes est le plus décisif.

Annexes

A. Hyperparamètres complets du modèle XGBoost

```
{
  "lambda":          1.128e-08,
  "alpha":           2.176e-08,
  "max_depth":        10,
  "eta":              0.01117,
  "subsample":         0.9996,
  "colsample_bytree": 0.8366,
  "min_child_weight": 1,
  "gamma":            4.808,
  "objective":         "reg:squarederror",
  "eval_metric":       "rmse",
  "tree_method":       "hist",
  "seed":              42
}
```

B. Organisation du dépôt Git

```
hackathon-wadi/
|-- data/
|   |-- WADI_14days_new.csv      # Donnees brutes
|   '-- WADI_clean.csv          # Donnees nettoyees (genere par EDA)
|-- notebooks/
|   |-- EDA_WADI.ipynb           # Analyse exploratoire
|   |-- WADI_XGBoost.ipynb       # Modelisation XGBoost
|   |-- deep_learning.ipynb      # TCN, LSTM, GRU
|   '-- baselines.ipynb         # ARIMA, ETS, persistence
|-- artefacts/
|   |-- best_params.json         # Hyperparametres Optuna
|   '-- metrics.json            # Metriques d'evaluation
'-- rapport/
    '-- rapport_WADI.tex        # Ce rapport
```

C. Glossaire

SCADA	Supervisory Control and Data Acquisition
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ACP	Analyse en Composantes Principales
TPE	Tree-structured Parzen Estimator (algorithme Optuna)
IQR	Interquartile Range (méthode Tukey)
TCN	Temporal Convolutional Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
PV	Process Value — valeur mesurée par le capteur
SP	Setpoint — consigne du régulateur
CO	Controller Output — commande envoyée à l'actionneur